

Rapport TER

Approche moderne du rendu différentiel

2026

Corentin VAILLANT, Julien LEFEBVRE

Université de Toulouse

2026

L3 MIDL

Encadrant : Mathias PAULIN

Table des matières

1	Introduction	3
2	Le Rendu	4
2.1	L'équation du rendu	4
2.2	Le Ray Tracing	4
2.3	Le Path Tracing	5
2.4	Autres Méthodes	8
3	La Différentiation	10
3.1	La Méthode Naïve	10
3.2	La Méthode Etudiée (Méthode de l'UC)	10
3.3	Autres Méthodes (Methode de l'EPFL)	10
4	Implementation du Path Tracer	11
5	Difficultées Rencontrées	12
6	Conclusion	13
7	Bibliographie	14

1 | Introduction

Le rendu différentiel est une branche de l'informatique graphique où l'on cherche à trouver la différentielle de l'image d'un rendu d'une scène, ce qui est très utile dans de nombreux domaines. Il possède de nombreuses applications, notamment dans le rendu plus classique, car il permet de faire varier des paramètres de la scène sans obligation de rendre la scène une nouvelle fois de zéro, ce qui peut être très coûteux. Il est aussi utile dans le rendu physiquement réaliste parce qu'il a permis de résoudre des problèmes d'analyse-par-synthèse dans des domaines comme le rendu de vêtements ou encore la création de matériaux translucides. Son utilité ne se limite pas qu'au rendu, il possède aussi des applications dans le domaine du machine learning car il permet d'entraîner des réseaux de neurones de manière plus efficace. Le rendu différentiel a une communauté de recherche très active. En effet ce domaine est très challengeant parce qu'à ce jour aucun algorithme efficace n'introduisant pas de biais n'a été découvert. Cela est notamment dû au fait de l'absence d'estimateurs de Monte Carlo efficaces.

Dans ce TER, nous nous sommes intéressés aux travaux de Cheng Zhang, Bailey Miller, Kai Yan, Ioannis Gkioulekas et Shuang Zhao (**Path-Space Differentiable Rendering**) qui proposent une solution sans biais à ce problème basée sur la séparation en deux sous-problèmes.

Pour effectuer ce TER, nous nous sommes aussi intéressés au ray tracing ainsi qu'au path tracing, notamment grâce à la série de livres **Raytracing in one weekend**, dans le but d'acquérir les bases nécessaires à la compréhension du papier.

Dans le cadre de ce travail, nous avons aussi implémenté un Path Tracer sur GPU avec Vulkan (**GitHub du projet**).

Nous allons introduire dans un premier temps le ray tracing ainsi que ses limites. Puis nous regarderons ce qu'est le Path Tracing et comment il corrige les limites du Ray Tracing. Dans un troisième temps, nous parlerons du rendu différentiel et de la méthode proposée par Shuang Zhao et son équipe. Enfin nous finirons par regarder une autre approche à ce problème avec les travaux de Tizian Zeltner, Sébastien Speierer, Iliyan Georgiev et Wenzel Jakob (**Monte Carlo Estimators for Differential Light Transport**).

2 | Le Rendu

2.1 | L'équation du rendu

Pour effectuer un rendu 3D, nous allons essayer de calculer la luminance des différentes surfaces de notre scène. La luminance représente la luminosité que l'œil humain perçoit, provenant d'une surface qui émet de la lumière soit en tant que source de lumière, soit par réflexion ou transmission. L'équation du rendu (**Page Wikipedia Equation du Rendu**) nous permet de calculer la luminance sur les différentes surfaces de notre scène 3D. On peut la définir comme ci-dessous :

Définition 2.1. (Equation du Rendu)

$$L_o(x, \omega_o) = L_e(x, \omega_o) + \int_{\Omega} L_i(x, \omega_i) f_i(x, \omega_0, \omega_i) \overline{\cos}(\theta_i) d\omega_i \quad (1)$$

- $L_o(x, \omega_o)$ représente la luminance sortante au point x dans la direction ω_o .
- $L_e(x, \omega_o)$ représente la luminance émise par le point x dans la direction ω_o .
- Ω représente la sphère de rayon 1 autour du point x .
- $L_i(x, \omega_i)$ représente la luminance arrivant en x depuis la direction ω_i et elle est définie de la manière suivante : $L_i(x, \omega_i) = L_o(vis(x, \omega_i), -\omega_i)$ où $vis(x, \omega_i)$ donne le premier point intersecté en partant dans la direction ω_i à partir du point x .
- $f_i(x, \omega_0, \omega_i)$ représente la BRDF qui indique comment la luminance arrivant en x depuis la direction ω_i et réfléchi dans la direction ω_o .
- θ représente l'angle formé ω_i et la $\vec{n}$ normale de la surface.
- $\overline{\cos}(\theta) = \cos(\theta)$ si $\cos(\theta) > 0$ sinon 0.

L'un des problèmes de cette équation est que l'on ne peut pas résoudre l'intégrale de façon analytique, notamment en raison de sa nature récursive. C'est pour cela que des méthodes pour estimer cette intégrale ont été mises en place.

2.2 | Le Ray Tracing

Définition 2.2. (Rayon) On appelle rayon un couple $\vec{r} = (o, \vec{d}) \in \mathbb{R}^3 \times \mathbb{U}(\mathbb{R}^3)$ où o est un point désignant l'origine du rayon et $\vec{d}$ est un vecteur unitaire désignant la direction du rayon. À noter que la direction n'est pas nécessairement unitaire, mais nous la prendrons ainsi sans perte de généralité pour des raisons de simplicité dans le reste de ce rapport.

Le ray tracing (lancée de rayon dans la langue de Molière) est un algorithme permettant d'estimer l'équation du rendu (éq. 1). Il se base sur une technique de lancer de rayon à partir de la caméra vers la scène, c'est-à-dire dans le sens inverse de la lumière. Cela donne le même résultat que dans le sens de la lumière d'après le principe du retour inverse de la lumière de Fermat. Il se base aussi sur les lois de Snell-Decartes de réflexion et réfraction de la lumière. Une méthode de ray tracing naïve de la marche aléatoire :

Algorithme 1 . – Marche Aléatoire

```

1: procédure  $L_i$ (Rayon  $\vec{r}$ , depth  $\in \mathbb{N}$ )
2:   intersection  $\leftarrow$  Intersection( $\vec{r}$ )
3:    $\triangleright$  Cas où le rayon n'intersecte aucune surface
4:   si Pas d'intersection alors
5:      $L_e \leftarrow 0$ 
6:     pour lumière  $\in$  Lumières Directionelles faire
7:        $L_e \leftarrow L_e + \text{lumière}.L_e(\vec{r})$ 
8:     fin
9:     retourner  $L_e$ 
10:  fin
11:   $\triangleright$  Recupération de la surface intersectée et de son émission
12:  surface  $\leftarrow$  intersection.surface
13:   $\omega_o \leftarrow -\vec{r}.\text{direction}$ 
14:   $L_e \leftarrow \text{surface}.L_e(\omega_o)$ 
15:   $\triangleright$  Cas où on a atteint le nombre maximum d'itération
16:  si depth = maxDepth alors
17:    retourner  $L_e$ 
18:  fin
19:   $\triangleright$  Calcul de la partie non récursive de l'intégrale de l'équation du rendu
20:  bsdf  $\leftarrow$  surface.BSDF
21:   $\omega_i \leftarrow \mathcal{U}(\Omega)$ 
22:   $f_{\cos} \leftarrow \text{bsdf}(\omega_o, \omega_i) \times |\omega_i \cdot \vec{n}|$ 
23:   $\triangleright$  Si la partie non récursive est nulle il est inutile de continuer d'itérer
24:  si  $f_{\cos} = 0$  alors
25:    retourner  $L_e$ 
26:  fin
27:   $\triangleright$  Appel récursif
28:   $\vec{r} \leftarrow \text{CreerRayon}(\omega_i)$ 
29:  retourner  $L_e + f_{\cos} \times L_i(\vec{r}, \text{depth}+1) / (1 / (4 \times \pi))$ 
30: fin

```

2.3 | Le Path Tracing

L'équation du rendu (éq. 1) peut être réécrite de façon à enlever son aspect récursif. Pour cela, au lieu d'intégrer sur la sphère unitée autour de chacun

des points, nous allons effectuer un changement de variable et intégrer sur des chemins. Cette réécriture a été proposée par Veach en 1997 notamment dans son papier **Metropolis Light Transport**.

Définition 2.3. (Chemin de lumière et espace des chemins) Soit $\mathcal{M}$ l'ensemble des surfaces des objets de notre scène. On appelle chemin de lumière (ou light path) un vecteur $\bar{x} = (x_0, x_1, \dots, x_N)$ de $\mathcal{M}^{N+1}$. Les chemins commencent sur une lumière en x_0 pour aller jusqu'à la caméra en x_N . On appelle espace des chemins (ou path space) l'ensemble $\Omega := \cup_{N=1}^{\infty} \mathcal{M}^{N+1}$.

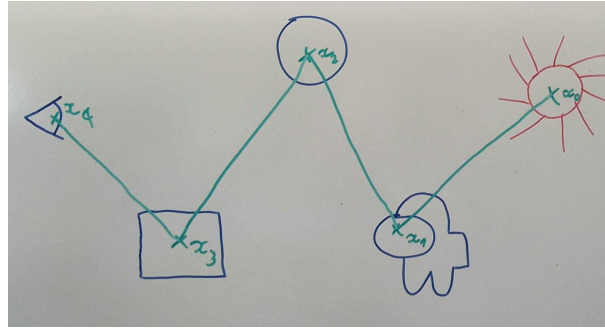


Fig. 1 . – Exemple de chemin de $\mathcal{M}^{N+1}$

En réécrivant l'équation du rendu (éq. 1) sur l'espace des chemins, on obtient:

Définition 2.4. (Équation du rendu sur l'espace des chemins)

$$I = \int_{\Omega} f(\bar{x}) d\mu(\bar{x}) \quad (2)$$

Où μ est la mesure du produit des aires défini par $d\mu(\bar{x}) := \prod_{n=0}^N dA(x_n)$ avec A la mesure de l'aire d'une surface et $f(\bar{x})$ est défini de la façon suivante :

$$f(\bar{x}) = \left(\prod_{n=0}^{N-1} g(x_{n+1} : x_n, w_n) \right) W_e(x_N \rightarrow x_{N-1}) \quad (3)$$

où W_e est l'importance du capteur dans notre cas, W_e sera une constante égale à 1 car on utilise une caméra trou d'épingle et

$$g(x_{n+1} : x_n, w_n) := f_s(x_n \rightarrow x_{n+1}) G(x_n \leftrightarrow x_{n+1}) \quad (4)$$

où f_s est la BSDF au point x_n dans la direction arrivant de x_{n-1} et allant vers x_{n+1} si $n > 0$ sinon $f_s := L_e(x_0 \rightarrow x_1)$ où L_e est l'émission de la surface x_0 dans la direction de x_1 et

$$G(x_n \leftrightarrow x_{n+1}) := \mathbb{V}(x_n \leftrightarrow x_{n+1}) G_0(x_n \leftrightarrow x_{n+1}) \quad (5)$$

où $\mathbb{V}(x_n \leftrightarrow x_{n+1})$ vaut 1 si x_n et x_{n+1} sont visibles, c'est-à-dire s'il n'y a pas de surfaces opaques entre eux, et 0 sinon et

$$G_0 := \frac{|\vec{n}_{x_n} \cdot \omega_n| |\vec{n}_{x_{n+1}} \cdot (-\omega_n)|}{\|x_{n+1} - x_n\|^2} \quad (6)$$

où ω_n représente le vecteur unitaire partant x_n et pointant vers x_{n+1} .

Cette réécriture de l'équation du rendu sur l'espace des chemins est ce qui nous permettra dans la suite, lorsque nous aborderons les travaux de Shung Zhao et son équipe (**Path-Space Differentiable Rendering**) de calculer la différentielle de notre scène. En effet, on verra qu'il est possible de « rentrer » la différentielle à l'intérieur de l'intégrale et que cela nous créera deux intégrales, une intérieure (ou interior) et une de bord (ou boundary).

Cette réécriture est aussi à la base de l'algorithme du path tracing qui est fondamental car nous verrons qu'il est possible de le différencier. Il se base sur une méthode de Monte Carlo pour estimer l'intégrale sur les chemins (éq. 2).

Algorithme 2 . – Path Tracer

```

1: procédure  $L_i(\text{Rayon } \vec{r})$ 
2:    $L \leftarrow 0$ 
3:    $\beta \leftarrow 1$ 
4:   specularBounce  $\leftarrow$  vrai
5:   depth  $\leftarrow 0$ 
6:   tant que  $\beta$  non nul faire
7:     intersection  $\leftarrow$  Intersection( $\vec{r}$ )
8:     si Pas d'intersection alors
9:       si specularBounce alors
10:        pour lumière  $\in$  Lumières Directionelles faire
11:           $L \leftarrow L + \text{lumière}.L_e(\vec{r})$ 
12:        fin
13:      BREAK
14:    fin
15:  fin
16:  surface  $\leftarrow$  intersection.surface
17:  si specularBounce alors
18:     $L \leftarrow L + \text{surface}.L_e(-\vec{r}.\text{direction})$ 
19:  fin
20:  si depth = maxDepth alors
21:    BREAK
22:  fin
23:  bsdf  $\leftarrow$  surface.BSDF( $\vec{r}$ )
24:  si bsdf = 0 alors
25:    passerIntersection( $\vec{r}$ , intersection.tTouche)
    Continuer
26:  fin
27:   $\omega_o \leftarrow \vec{r}$ 

```

```

28:   lumière ← ChoisirUneLumiereAléatoirement()
29:   si lumière != 0 alors
30:     pointLumière ← lumière.ChoisirPointAléatoirement()
31:     ls ← lumière.echantillonner $L_i$ (intersection,pointLumière)
32:     si ls != 0 et ls.pdf > 0 alors
33:        $\omega_i \leftarrow ls.\omega_i$ 
34:        $f_{\cos} \leftarrow \text{bsdf}.f(\omega_o, \omega_i) \times |\omega_i \cdot \vec{n}|$ 
35:       si f != 0 et Visible(intersection,ls.pLumière) alors
36:          $L \leftarrow L + \beta \times f_{\cos} \times ls.L / (\text{lumière}.p \times ls.pdf)$ 
37:       fin
38:     fin
39:   fin
40:   bs ← bsdf.echantillonner( $\omega_o$ )
41:   si bs = 0 alors
42:     BREAK
43:   fin
44:    $\beta \leftarrow \beta \times bs.f \times |\omega_i \cdot \vec{n}| / bs.pdf$ 
45:   specularBounce ← bs.estSpeculaire
46:    $\vec{r} \leftarrow \text{CreerRayon}(bs.\omega_i)$ 
47: fin
48: retourner L
49: fin

```

2.4 | Autres Méthodes

De nombreuses autres méthodes existent pour résoudre l'équation du rendu et ont chacune leurs avantages mais aussi leurs inconvénients. Parmi celles-ci, on retrouve :

1. La rasterisation qui, comme le ray tracing, est une quadrature de l'équation du rendu. Elle est non biaisée et différentiable mais elle reste très limitée, notamment au niveau de la différenciation et à une convergence assez lente.
2. Le photon mapping qui se base aussi sur une technique de lancer de rayon. Contrairement au ray tracing et au path tracing, les rayons sont envoyés depuis les lumières et on enregistre leurs chemins dans une photon map. Puis pour effectuer le rendu, on va envoyer un rayon depuis la caméra et à son point d'intersection avec une surface, regarder la concentration de points à proximité sur la photon map avec un algorithme des plus proches voisins. Cet algorithme est biaisé, donc il n'a pas vraiment d'intérêt à être différencié, mais il joue un rôle dans la méthode proposée par Shuang Zhao et son équipe (**Path-Space Differentiable Rendering**), nous en reparlerons dans la section dédiée.

3. Le bidirectional path tracing est une version améliorée du path tracing. Au lieu de créer des chemins uniquement depuis la caméra, il va aussi créer des chemins partant des lumières et les connecter entre eux avec un rayon de visibilité. De plus cet algorithme est non biaisé mais, malheureusement, à ce jour aucune technique de différentiation n'a été découverte.
4. Bien d'autres méthodes existent comme le **Metropolis Ligth Transport** qui se base sur l'algorithme de Metropolis-Hastings, le **Vertex Connection and Merging** qui fusionne le bidirectional path tracing et le photon mapping, le **Cone Tracing** qui donne une épaisseur aux rayons, ce que ne fait pas le raytracing ou encore le **Gaussian Splating** qui effectue le rendu à l'aide de données extraites d'image et beaucoup d'autres...

3 | La Différentiation

3.1 | La Méthode Naïve

3.2 | La Méthode Etudiée (Méthode de l'UC)

3.3 | Autres Méthodes (Méthode de l'EPFL)

4 | Implementation du Path Tracer

5 | Difficultés Rencontrées

6 | Conclusion

7 | Bibliographie